LAB 10:

Question#01:

Code:

```cpp
#include <iostream>
using namespace std;

template <typename T1, typename T2>
class Calculator
{
    T1 num1;
    T2 num2;

public:
    Calculator(T1 n1, T2 n2)
    {
        num1 = n1;
        num2 = n2;
    }

    void add()
    {
        cout << "Addition: " << num1 + num2 << endl;
    }

    void subtract()
    {
        cout << "Subtraction: " << num1 - num2 << endl;
    }

    void multiply()
    {
        cout << "Multiplication: " << num1 * num2 << endl;
    }

    void divide()
    {
        if (num2 != 0)
            cout << "Division: " << (double)num1 / num2 << endl;
        else
            cout << "Division by zero is not allowed!" << endl;
    }
```

```
};

int main()
{

    cout << "=== Int & Int ===" << endl;
    Calculator<int, int> c1(10, 3);
    c1.add();
    c1.subtract();
    c1.multiply();
    c1.divide();

    cout << "\n=== Int & Double ===" << endl;
    Calculator<int, double> c2(10, 3.5);
    c2.add();
    c2.subtract();
    c2.multiply();
    c2.divide();

    return 0;
}
```

Output:

```
=== Int & Int ===
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.33333

=== Int & Double ===
Addition: 13.5
Subtraction: 6.5
Multiplication: 35
Division: 2.85714
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 10>
```

Question#02:

Code:

```cpp
#include <iostream>
using namespace std;

// Template function for swapping same datatype
template <typename T>
void swapValues(T &a, T &b)
{
    T temp = a;
    a = b;
    b = temp;
}

// Template function for swapping different datatypes
template <typename T1, typename T2>
void swapValues(T1 &a, T2 &b)
{
    T1 temp = a;
    a = (T1)b;
    b = (T2)temp;
}

int main()
{

    cout << "=== Int & Int ===" << endl;
    int x = 10, y = 20;
    cout << "Before Swap: x = " << x << ", y = " << y << endl;
    swapValues(x, y);
    cout << "After Swap:  x = " << x << ", y = " << y << endl;

    cout << "\n=== Double & Double ===" << endl;
    double a = 3.14, b = 7.77;
    cout << "Before Swap: a = " << a << ", b = " << b << endl;
    swapValues(a, b);
    cout << "After Swap:  a = " << a << ", b = " << b << endl;

    cout << "\n=== Int & Double ===" << endl;
    int p = 5;
    double q = 9.99;
    cout << "Before Swap: p = " << p << ", q = " << q << endl;
    swapValues(p, q);
    cout << "After Swap:  p = " << p << ", q = " << q << endl;

    return 0;
}
```

Output:

```
=== Int & Int ===
Before Swap: x = 10, y = 20
After Swap:  x = 20, y = 10

=== Double & Double ===
Before Swap: a = 3.14, b = 7.77
After Swap:  a = 7.77, b = 3.14

=== Int & Double ===
Before Swap: p = 5, q = 9.99
After Swap:  p = 9, q = 5
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 10>
```

Question#03:

Code:

```cpp
#include <iostream>
using namespace std;

template <typename T>
class mycontainer {
    T element;

public:
    mycontainer(T el) {
        element = el;
    }

    void increase() {
        element++;
        cout << "Increased Value: " << element << endl;
    }
};

// Template specialization for char
template <>
class mycontainer<char> {
    char element;
```

```
public:
    mycontainer(char el) {
        element = el;
    }

    void uppercase() {
        if (element >= 'a' && element <= 'z')
            element = element - 32; // converts to uppercase
        cout << "Uppercase: " << element << endl;
    }
};

int main() {

    cout << "=== Int Container ===" << endl;
    mycontainer<int> c1(5);
    c1.increase();


    cout << "\n=== Double Container ===" << endl;
    mycontainer<double> c2(3.14);
    c2.increase();


    cout << "\n=== Char Container (Specialization) ===" << endl;
    mycontainer<char> c3('h');
    c3.uppercase();

    return 0;
}
```

Output:

```
=== Int Container ===
Increased Value: 6

=== Double Container ===
Increased Value: 4.14

=== Char Container (Specialization) ===
Uppercase: H
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 10>
```

Question#04:

Code:

```cpp
#include <iostream>
using namespace std;

// Abstract class template for 1D dynamic array
template <typename T>
class AbstractArray {
public:
    virtual bool isFull() = 0;
    virtual bool isEmpty() = 0;
    virtual int size() = 0;
    virtual T Front() = 0;
    virtual T Rear() = 0;
    virtual void enqueue(T val) = 0;
    virtual void dequeue() = 0;
    virtual void resize() = 0;

    virtual ~AbstractArray() {}
};

// Derived Queue class template
template <typename T>
class Queue : public AbstractArray<T> {
    T* arr;
    int capacity;
    int count;
    int front;
    int rear;

public:
    Queue(int cap = 5) {
        capacity = cap;
        arr = new T[capacity];
        count = 0;
        front = 0;
        rear = -1;
    }

    bool isFull() override {
```

```
        return count == capacity;
    }

    bool isEmpty() override {
        return count == 0;
    }

    int size() override {
        return count;
    }

    T Front() override {
        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return T();
        }
        return arr[front];
    }

    T Rear() override {
        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return T();
        }
        return arr[rear];
    }

    void enqueue(T val) override {
        if (isFull()) {
            cout << "Queue is full! Resizing..." << endl;
            resize();
        }
        rear = (rear + 1) % capacity;
        arr[rear] = val;
        count++;
        cout << val << " enqueued successfully." << endl;
    }

    void dequeue() override {
        if (isEmpty()) {
            cout << "Queue is empty! Cannot dequeue." << endl;
            return;
        }
        cout << arr[front] << " dequeued successfully." << endl;
        front = (front + 1) % capacity;
```

```
        count--;
    }

    void resize() override {
        int newCapacity = capacity * 2;
        T* newArr = new T[newCapacity];

        for (int i = 0; i < count; i++)
            newArr[i] = arr[(front + i) % capacity];

        delete[] arr;
        arr = newArr;
        front = 0;
        rear = count - 1;
        capacity = newCapacity;
        cout << "Queue resized to capacity: " << capacity << endl;
    }

    void display() {
        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return;
        }
        cout << "Queue: ";
        for (int i = 0; i < count; i++)
            cout << arr[(front + i) % capacity] << " ";
        cout << endl;
    }

    ~Queue() {
        delete[] arr;
    }
};

int main() {
    cout << "=== Integer Queue ===" << endl;
    Queue<int> q(3); // capacity of 3

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
    q.display();

    cout << "Front: " << q.Front() << endl;
    cout << "Rear: " << q.Rear() << endl;
```

```
    cout << "Size: " << q.size() << endl;

    // This will trigger resize
    q.enqueue(40);
    q.display();

    q.dequeue();
    q.dequeue();
    q.display();

    cout << "\n=== String Queue ===" << endl;
    Queue<string> q2(2);
    q2.enqueue("Ali");
    q2.enqueue("Sara");
    q2.display();
    q2.enqueue("Ahmed"); // triggers resize
    q2.display();

    return 0;
}
```

Output:

```
=== Integer Queue ===
10 enqueued successfully.
20 enqueued successfully.
30 enqueued successfully.
Queue: 10 20 30
Front: 10
Rear: 30
Size: 3
Queue is full! Resizing...
Queue resized to capacity: 6
40 enqueued successfully.
Queue: 10 20 30 40
10 dequeued successfully.
20 dequeued successfully.
Queue: 30 40

=== String Queue ===
Ali enqueued successfully.
Sara enqueued successfully.
Queue: Ali Sara
Queue is full! Resizing...
Queue resized to capacity: 4
Ahmed enqueued successfully.
Queue: Ali Sara Ahmed
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 10>
```

Question#05:

Code:

```cpp
#include <iostream>
#include <string>
using namespace std;

template <typename T>
class AbstractArray
{
public:
    virtual bool isFull() = 0;
    virtual bool isEmpty() = 0;
    virtual int size() = 0;
    virtual T Front() = 0;
    virtual T Rear() = 0;
    virtual void enqueue(T val) = 0;
    virtual void dequeue() = 0;
    virtual void resize() = 0;
    virtual ~AbstractArray() {}
};

template <typename T>
class Queue : public AbstractArray<T>
{
    T *arr;
    int capacity;
    int count;
    int front;
    int rear;

public:
    Queue(int cap = 5)
    {
        capacity = cap;
        arr = new T[capacity];
        count = 0;
        front = 0;
        rear = -1;
    }
```

```
    bool isFull() override { return count == capacity; }
    bool isEmpty() override { return count == 0; }
    int size() override { return count; }

    T Front() override
    {
        if (isEmpty())
        {
            cout << "Queue is empty!" << endl;
            return T();
        }
        return arr[front];
    }

    T Rear() override
    {
        if (isEmpty())
        {
            cout << "Queue is empty!" << endl;
            return T();
        }
        return arr[rear];
    }

    void enqueue(T val) override
    {
        if (isFull())
            resize();
        rear = (rear + 1) % capacity;
        arr[rear] = val;
        count++;
    }

    void dequeue() override
    {
        if (isEmpty())
        {
            cout << "Queue is empty!" << endl;
            return;
        }
        front = (front + 1) % capacity;
        count--;
    }

    void resize() override
```

```
    {
        int newCapacity = capacity * 2;
        T *newArr = new T[newCapacity];
        for (int i = 0; i < count; i++)
            newArr[i] = arr[(front + i) % capacity];
        delete[] arr;
        arr = newArr;
        front = 0;
        rear = count - 1;
        capacity = newCapacity;
    }

    ~Queue() { delete[] arr; }
};

// ---- Print Shop Class ----
class PrintShop
{
    Queue<string> jobQueue;
    bool printerIdle;

public:
    PrintShop() : jobQueue(5)
    {
        printerIdle = true;
    }

    // Add a print job to the queue
    void addJob(string job)
    {
        cout << "Job received: \"" << job << "\" added to queue." << endl;
        jobQueue.enqueue(job);

        // If printer is idle, start printing immediately
        if (printerIdle)
            processJob();
    }

    // Process the next job in queue
    void processJob()
    {
        if (!jobQueue.isEmpty())
        {
            printerIdle = false;
            string job = jobQueue.Front();
```

```
            jobQueue.dequeue();
            cout << "Printing: \"" << job << "\"..." << endl;
            cout << "\"" << job << "\" completed!" << endl;

            // Check if more jobs exist
            if (!jobQueue.isEmpty())
            {
                cout << "Next job found in queue. Continuing..." << endl;
                processJob(); // process next job
            }
            else
            {
                printerIdle = true;
                cout << "No more jobs. Printer is now idle." << endl;
            }
        }
    }
};

int main()
{
    PrintShop shop;

    cout << "===== Print Shop Simulation =====" << endl
         << endl;

    shop.addJob("Resume.pdf");
    cout << endl;
    shop.addJob("Assignment.docx");
    cout << endl;
    shop.addJob("Invoice.pdf");
    cout << endl;
    shop.addJob("Report.docx");

    return 0;
}
```

Output:

```
===== Print Shop Simulation =====

Job received: "Resume.pdf" added to queue.
Printing: "Resume.pdf"...
"Resume.pdf" completed!
No more jobs. Printer is now idle.

Job received: "Assignment.docx" added to queue.
Printing: "Assignment.docx"...
"Assignment.docx" completed!
No more jobs. Printer is now idle.

Job received: "Invoice.pdf" added to queue.
Printing: "Invoice.pdf"...
"Invoice.pdf" completed!
No more jobs. Printer is now idle.

Job received: "Report.docx" added to queue.
Printing: "Report.docx"...
"Report.docx" completed!
No more jobs. Printer is now idle.
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 10>
```